2 Water Jug Problem

Aim / Objective

1. To understand the concept of state space search in Artificial Intelligence.
2. To implement the Water Jug Problem using a procedural approach.
3. To apply basic control flow and conditional logic in Python programming.

Theory

The Water Jug Problem is a classic problem in Artificial Intelligence used to illustrate state space search. The problem typically involves two jugs with different capacities, and the goal is to measure out a specific amount of water using only these two jugs and an unlimited supply of water. The problem starts with both jugs empty, and a set of permissible operations, such as filling a jug, emptying a jug, or pouring water from one jug to the other, must be followed.

Each combination of water levels in the two jugs represents a *state* in the problem. The set of all possible states forms the *state space*. The permissible operations are the *operators* that transition the problem from one state to another. The objective is to find a sequence of valid operations (a path) from the initial state (both jugs empty) to the *goal state* (one of the jugs contains the target amount of water). This problem is fundamental for introducing concepts like state representation, goal testing, and search strategies in a simplified, yet effective, manner. For this particular implementation, the state space is explored manually through user interaction, rather than an automated search algorithm, to better understand the impact of each operation on the state. The chosen capacities for Jug X and Jug Y are 3L and 5L, respectively, with a goal of achieving 4L.

Algorithm / Methodology

1. Define a function, `function1()`, to encapsulate the logic for the Water Jug Problem.
2. Initialize the capacities of Jug X (`xc` = 3) and Jug Y (`yc` = 5), and the target amount (`final_check` = 4).
3. Initialize the current water level in Jug X (`x` = 0) and Jug Y (`y` = 0).
4. Start an infinite loop (`while True`) to repeatedly display the current state and prompt the user for an action.
5. Inside the loop, display the current water levels in both jugs and the goal state.
6. Check if the goal state is reached (if `x` equals `final_check` or `y` equals `final_check`). If reached, print a success message and break the loop.

7. Display the menu of available operations:
    a. transfer X to Y
    b. transfer Y to X
    c. fill X
    d. fill Y
    e. empty X
    f. empty Y
    g. exit
8. Prompt the user to select an operation (1-7).
9. Use conditional statements (`if/elif/else`) to execute the chosen operation:
    - **Choice 1 (Transfer X to Y):** Calculate the amount to transfer as the minimum of the water in X and the remaining capacity in Y. Update y and x.
    - **Choice 2 (Transfer Y to X):** Calculate the amount to transfer as the minimum of the water in Y and the remaining capacity in X. Update x and y.
    - **Choice 3 (Fill X):** Set x to xc.
    - **Choice 4 (Fill Y):** Set y to yc.
    - **Choice 5 (Empty X):** Set x to 0.
    - **Choice 6 (Empty Y):** Set y to 0.
    - **Choice 7 (Exit):** Print a closing message and break the loop.
    - **Other:** Print an error message for an invalid selection.
10. Call `function1()` to start the simulation.

Dataset Description (Source, Attributes, Size)

Source:

The dataset is user-generated during program execution and does not come from an external source. Each state is formed based on user-selected operations on the water jugs.

Attributes:

Water level in Jug X (0–3 liters)
Water level in Jug Y (0–5 liters)
Operation performed
Goal state status (achieved or not)

Dataset Size:

The dataset size is dynamic and depends on the number of user actions. The maximum possible state space consists of 24 discrete states.

Nature:

Discrete, finite, and state-based dataset used to demonstrate state space search in Artificial Intelligence.

Code

```
def function1():

 xc = 3

 yc = 5

 final_check = 4

 x = 0

 y = 0

 while True:

  print(f"Live: Jug X = {x}L, Jug Y = {y}L (Goal: {final_check}L)")

  if x==final_check or y==final_check:

   print("Happyyy!! you got the answerr")
```

```
 break
print("1. transfer X to Y")
print("2. transfer Y to X")
print("3. fill X")
print("4. fill Y")
print("5. empty X")
print("6. empty Y")
print("7. exit")
choice = int(input("\nSelect from 1-7: "))
if choice == 1:
 transfer = min(x,yc-y)
 y = y + transfer
 x = x - transfer
elif choice == 2:
 transfer = min(y,xc-x)
 x = x + transfer
 y = y - transfer
elif choice==3:
 x = xc
elif choice==4:
 y = yc
elif choice==5:
```

```
 x = 0
elif choice==6:
 y = 0
elif choice==7:
 print("Closing!")
 break
else:
 print("Wrong selected!")
function1()
```

OUTPUT:

```
Run   GamifyMain
C:\Users\ADARSH\AppData\Local\Programs\Python\Python313\python.exe C:\Users\ADARSH\OneDrive\Desktop\AI_LAB\GamifyMain.py
Live: Jug X = 0L, Jug Y = 0L (Goal: 4L)
1. transfer X to Y
2. transfer Y to X
3. fill X
4. fill Y
5. empty X
6. empty Y
7. exit

Select from 1-7: 4
Live: Jug X = 0L, Jug Y = 5L (Goal: 4L)
1. transfer X to Y
2. transfer Y to X
3. fill X
4. fill Y
5. empty X
6. empty Y
7. exit

Select from 1-7: 2
Live: Jug X = 3L, Jug Y = 2L (Goal: 4L)
1. transfer X to Y
2. transfer Y to X
3. fill X
4. fill Y
```

```
Run    GamifyMain
4. fill Y
5. empty X
6. empty Y
7. exit

Select from 1-7: 5
Live: Jug X = 0L, Jug Y = 2L (Goal: 4L)
1. transfer X to Y
2. transfer Y to X
3. fill X
4. fill Y
5. empty X
6. empty Y
7. exit

Select from 1-7: 2
Live: Jug X = 2L, Jug Y = 0L (Goal: 4L)
1. transfer X to Y
2. transfer Y to X
3. fill X
4. fill Y
5. empty X
6. empty Y
7. exit

Select from 1-7: 4
```

```
Run    GamifyMain
Select from 1-7: 2
Live: Jug X = 2L, Jug Y = 0L (Goal: 4L)
1. transfer X to Y
2. transfer Y to X
3. fill X
4. fill Y
5. empty X
6. empty Y
7. exit

Select from 1-7: 4
Live: Jug X = 2L, Jug Y = 5L (Goal: 4L)
1. transfer X to Y
2. transfer Y to X
3. fill X
4. fill Y
5. empty X
6. empty Y
7. exit

Select from 1-7: 2
Live: Jug X = 3L, Jug Y = 4L (Goal: 4L)
Happyyy!! you got the answerr

Process finished with exit code 0
```

Result Analysis (Interpretation of Output)

The program successfully simulates the Water Jug Problem using user-driven state transitions.
At each step, the current water levels of Jug X and Jug Y are displayed, allowing clear tracking of state changes.
Valid operations such as filling, emptying, and transferring water correctly modify the jug states.
The system continuously checks for the goal condition and terminates execution once 4 liters is achieved in either jug.
This confirms that the procedural logic and state space representation work as expected.

Conclusion

The Water Jug Problem was implemented successfully using a procedural approach in Python.
The experiment helped in understanding state representation, state transitions, and goal testing in Artificial Intelligence.
Manual exploration of the state space provided better insight into how different operations affect the system state.
Overall, the program effectively demonstrates the concept of state space search in a simple and interactive manner.

File
Calendar event